

🔗 Hai ô markdown đầu tiên giới thiệu về mục tiêu của dự án: **chuyển đổi mã Python sang C++ hiệu suất cao** bằng cách sử dụng **mô hình Frontier** như GPT-4o hoặc Claude-3.5.

🔗 Ngoài ra, phần markdown cũng nhắc nhở người dùng **cập nhật mã nguồn mới nhất** từ GitHub để đảm bảo sử dụng các bản cập nhật mới nhất.

```
import os
import io
import sys
from dotenv import load_dotenv
from openai import OpenAI
import google.generativeai
import anthropic
from IPython.display import Markdown, display, update_display
import gradio as gr
import subprocess
```

- **dotenv**: Đọc các biến môi trường từ tệp .env, giúp bảo mật thông tin API.
- **openai, anthropic, google.generativeai**: Sử dụng API của OpenAI (GPT-4o), Claude (Anthropic), và Google Gemini để tạo mã C++.
- **gradio**: Tạo giao diện web để chạy mô hình dễ dàng.
- **subprocess**: Dùng để chạy các lệnh hệ thống, như biên dịch và chạy mã C++.

```
load_dotenv(override=True)
os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY', 'your-key-if-not-using-env')
os.environ['ANTHROPIC_API_KEY'] = os.getenv('ANTHROPIC_API_KEY', 'your-key-if-not-using-env')
```

Tải biến môi trường từ tệp .env và đặt giá trị API key cho OpenAI và Anthropic.

```
openai = OpenAI()
claude = anthropic.Anthropic()
OPENAI_MODEL = "gpt-4o"
CLAUDE_MODEL = "claude-3-5-sonnet-20240620"
```

🔗 Khởi tạo hai mô hình AI:

- **GPT-4o** của OpenAI
- **Claude 3.5 Sonnet** của Anthropic

🔗 Người dùng có thể chọn **mô hình rẻ hơn** bằng cách bỏ comment dòng dưới:

```
# OPENAI_MODEL = "gpt-4o-mini"
```

```
# CLAUDE_MODEL = "claude-3-haiku-20240307"
```

Xác định yêu cầu rõ ràng cho AI:

Chuyển đổi Python sang C++

Chỉ trả về mã C++ (không có giải thích)

Ưu tiên hiệu suất cao trên Mac M1

- *system_message = "You are an assistant that reimplements Python code in high performance C++ for an M1 Mac. "*
- *system_message += "Respond only with C++ code; use comments sparingly and do not provide any explanation other than occasional comments. "*
- *system_message += "The C++ response needs to produce an identical output in the fastest possible time."*

Hàm tạo prompt cho người dung:

```
def user_prompt_for(python):  
    user_prompt = "Rewrite this Python code in C++ with the fastest possible implementation t  
    user_prompt += "Respond only with C++ code; do not explain your work other than a few com  
    user_prompt += "Pay attention to number types to ensure no int overflows. Remember to #in  
    user_prompt += python  
    return user_prompt
```

Hàm này tạo prompt yêu cầu AI chuyển đổi mã Python sang C++, với lưu ý:

- Tối ưu tốc độ thực thi
- Tránh tràn số nguyên (int overflow)
- Bao gồm thư viện C++ cần thiết như `io` và `manip`

Hàm tạo danh sách tin nhắn cho AI:

```
def messages_for(python):
    return [
        {"role": "system", "content": system_message},
        {"role": "user", "content": user_prompt_for(python)}
    ]
```

Tạo danh sách tin nhắn gồm:

- Tin nhắn hệ thống: Định nghĩa vai trò của AI.
- Tin nhắn người dùng: Yêu cầu AI chuyển đổi mã Python sang C++.

```
def write_output(cpp):
    code = cpp.replace("```cpp", "").replace("```", "")
    with open("optimized.cpp", "w") as f:
        f.write(code)
```

Xóa định dạng markdown của mã C++ (````cpp` và `````).

Ghi mã C++ vào tệp `optimized.cpp` để có thể biên dịch và chạy.

```
def optimize_gpt(python):
    stream = openai.chat.completions.create(model=OPENAI_MODEL, messages=messages_for(python))
    reply = ""
    for chunk in stream:
        fragment = chunk.choices[0].delta.content or ""
        reply += fragment
        print(fragment, end='', flush=True)
    write_output(reply)
```

Gửi yêu cầu đến **GPT-4o** để chuyển đổi mã Python sang C++.

Nhận dữ liệu theo **dòng (stream)** để tăng tốc độ phản hồi.

Lưu kết quả vào **optimized.cpp**.

```
def optimize_claude(python):
    result = claude.messages.stream(
        model=CLAUDE_MODEL,
        max_tokens=2000,
        system=system_message,
        messages=[{"role": "user", "content": user_prompt_for(python)}],
    )
    reply = ""
    with result as stream:
        for text in stream.text_stream:
            reply += text
            print(text, end="", flush=True)
    write_output(reply)
```

Gửi yêu cầu đến **Claude-3.5-Sonnet** để chuyển đổi mã Python sang C++.

Nhận dữ liệu **dưới dạng dòng** để in ra kết quả ngay lập tức.

Lưu kết quả vào **optimized.cpp**.

```
exec(pi)
optimize_gpt(pi)
exec(python_hard)
optimize_gpt(python_hard)
```

exec(pi): Chạy mã Python tính toán **số Pi**.

optimize_gpt(pi): Yêu cầu GPT-4o **tối ưu hóa mã Python thành C++**.

Tương tự cho python_hard (một bài toán khó hơn về **tổng dãy con lớn nhất**).

```
!clang++ -O3 -std=c++17 -march=armv8.3-a -o optimized optimized.cpp
!./optimized
```

Biên dịch optimized.cpp thành tệp thực thi optimized với tối ưu hóa -O3.

Chạy chương trình optimized và hiển thị kết quả.

